

Quiz 1

315832025 連怡卉

AI Tools: Gemini

我想從Quiz1著手了，我知道OpenCV的是BGR，我也想先實作OpenCV

沒錯，OpenCV 讀取彩色影像時預設是 BGR 順序，這在後續搭配 Matplotlib 顯示或使用 NumPy 自行計算時是個很關鍵的細節。

讓我們先從 基礎部分（OpenCV 實作灰階轉換）開始。在 `week1/main.ipynb` 中，一般流程包含三個主要動作：

1. 📂 讀取影像：使用 `cv2.imread()` 載入檔案路徑（例如放在 `Data/` 資料夾中的圖片）。
2. 🔄 轉換色彩空間：使用 `cv2.cvtColor()` 函式將 BGR 轉為灰階。
3. 🖼️ 呈現結果：使用 `matplotlib.pyplot` 或 `cv2.imshow()` 顯示原圖與灰階圖。

在呼叫 `cv2.cvtColor(img, code)` 時，你知道要將 BGR 轉為灰階，應該填入哪一個色彩轉換常數（`code`）嗎？

👉 🗨️ 📄 ...

我查了一下：

```
import cv2
img = cv2.imread('Data/nightmarket.jpg')
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

這個寫法完全正確！`cv2.COLOR_BGR2GRAY` 正是 OpenCV 用來將 BGR 轉為灰階的標準轉換碼。

在把轉換後的結果展示出來之前，我們可以先用 `matplotlib.pyplot` 來把原圖與灰階圖並排呈現。

這裡有一個初學者常遇到的小陷阱：

如果我們直接把 OpenCV 讀進來的 `img` 丟給 Matplotlib 的 `plt.imshow(img)`，畫面顏色看起來會偏藍（因為 Matplotlib 預期接收的是 RGB 格式）。

若要在 Matplotlib 正常顯示彩色原圖，你覺得在繪製前應該對 `img` 做什麼處理？另外，顯示灰階影像 `gray_img` 時，`plt.imshow()` 需要額外加上哪個參數，才不會被預設的熱度圖（綠黃紫彩色）影響呢？

👉 🗨️ 📄 ...

`cv2.COLOR_BGR2RGB`，不過我如果是直接執行ipynb是不是就不用直接開一個視窗了？

沒錯！使用 `cv2.COLOR_BGR2RGB` 轉成 RGB 後再丟給 Matplotlib 顯示，色彩就會完全正常。

關於視窗的問題：是的，在 Jupyter Notebook（.ipynb）裡面完全不需要彈出新視窗！

- 🖼️ 傳統視窗（`cv2.imshow()`）：會跳出一個獨立的作業系統視窗，在 Notebook 裡常常容易卡死（若沒正確處理 `cv2.waitKey` 與 `cv2.destroyAllWindows`），且重開 Notebook 後結果就消失了。
- 📄 Notebook 內嵌（`matplotlib.pyplot`）：只要呼叫 `plt.show()`，圖表與影像就會直接內嵌在 Notebook 單元格的下方，並永久保存在檔案裡，非常適合作為作業展示與報告！

彩色影像有三個通道： B 、 G 、 R ，若我們將這三個數值融合成一個灰階值，你直覺會想用什麼樣的數學方式來計算？是一般平均，還是加權平均呢？

我知道有一個HSV還是可以轉成那個然後把飽和度拉掉？

這是一個非常敏銳且很有創意的直覺！👉

在 HSV 色彩空間（Hue 色相、Saturation 飽和度、Value 明度）中：

- 飽和度（S）降為 0：代表完全沒有色彩純度，影像確實會變成灰色調。
- 明度（V）的算法：在 HSV 定義中， $V = \max(R, G, B)$ ，也就是直接取三個通道裡面的最大值。

如果把飽和度拉到 0，每個像素的數值就會直接等於 $\max(R, G, B)$ 。

不過，題目要我們和 OpenCV 的 `cvtColor_BGR2GRAY` 比較轉換結果。人眼對不同顏色的敏感度的其實大不相同（例如看到純綠色 會覺得比純藍色 亮得多）。

既然人眼對綠色特別敏感、對藍色相對較鈍，如果我們不用 $\max(R, G, B)$ ，而是用加權的方式來算灰階值：你覺得 R 、 G 、 B 這三個顏色，誰應該分配到最高的權重呢？

那應該是綠色的

沒錯，綠色正是權重最高的通道。

在廣泛使用的標準（例如 ITU-R BT.601，也就是 OpenCV `cvtColor` 預設採用的轉換標準）中，三個通道的權重分配如下：

$$Gray = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

可以看到，綠色（G）佔了將近 59% 的權重，而藍色（B）只有約 11%。

現在我們準備在 Code/ 資料夾下建立一個名為 `grayscale.py` 的檔案，用 NumPy 來實作這個公式。

在寫程式之前，請留意兩個細節：

- 影像陣列的形狀是 $(H, W, 3)$ ，且在 OpenCV 讀入時順序是 BGR。也就是說：
 - `img[:, :, 0]` 是哪一個顏色？
 - `img[:, :, 1]` 是哪一個顏色？
 - `img[:, :, 2]` 是哪一個顏色？
- 原始讀取的像素型態是 `uint8`（0 到 255 的整數）。如果直接乘以小數（浮點數）相加，最後存回灰階影像時，型態需要轉回什麼？

你能試著對應出這三個通道的順序嗎？

👁️ 🤖 📄 ➡️

依照老師的要求如下：- 使用 NumPy 自行實作灰階轉換演算法，並與 OpenCV 提供的方法進行比較
- 建議重複執行 N 次並取平均值，比較 (1) 執行速度、(2) 轉換結果、(3) 兩種方法之間的差異

為了完整達成老師的要求，我們將這項任務拆解成三個核心部分：

- 實作演算法：在 `Code/grayscale.py` 中用 NumPy 寫出灰階轉換函式。
- 效能與差異實驗：在 `main.ipynb` 中重複執行 N 次測量時間，並計算兩者的數值差異。
- 視覺化與分析：呈現結果圖、差異熱度圖（Difference Map），並以文字說明比較心得。

第一步：NumPy 灰階轉換函式

回顧剛才的權重公式：

$$Gray = 0.114 \times B + 0.587 \times G + 0.299 \times R$$

因為 OpenCV 讀入的陣列通道順序是 BGR，索引分別對應：

- `img[:, :, 0]` → B (藍)
- `img[:, :, 1]` → G (綠)
- `img[:, :, 2]` → R (紅)

我們可以在 Code/ 資料夾中建立 `grayscale.py`，寫入如下函式：

```
Python
import numpy as np

def bgr_to_gray_numpy(img: np.ndarray) -> np.ndarray:
    """使用 NumPy 依照 BT.601 公式將 BGR 影像轉換為灰階影像"""
    b = img[:, :, 0]
    g = img[:, :, 1]
    r = img[:, :, 2]

    # 浮點數加權演算
    gray = 0.114 * b + 0.587 * g + 0.299 * r

    # 因值丟入並轉回 8 位元無符號數 (0-255)
    return np.round(gray).astype(np.uint8)
```

第二步：設計實驗（在 `main.ipynb`）

老師列出的三個比較重點，我們可以這樣規劃：

評比項目	實作方式 依	觀察重點
(1) 執行速度	使用 <code>time.perf_counter()</code> 迴圈執行 N 次（例如 $N = 100$ ）並取平均值	OpenCV (C++ 底層優化) 與 NumPy 陣列運算的效能差距
(2) 轉換結果	用 Matplotlib 並排呈現兩者的灰階成品	肉眼觀察是否有明顯明暗或細節落差
(3) 兩者差異	計算絕對差值矩陣： <code>diff = np.abs(gray_cv.astype(int) - gray_np.astype(int))</code>	最大像素差值、平均差值 (MAE)，並探討四捨五入或定點數運算的微小差異

1. 差異比較的原理 📖

影像本質上是一個二維數值矩陣。假設 OpenCV 的灰階影像為矩陣 A ，你實作的灰階影像為矩陣 B ：

- 🔴 逐像素絕對差值 (Absolute Difference)：
比較每個對應位置 (x, y) 的亮度差距：

$$\text{Diff}(x, y) = |A(x, y) - B(x, y)|$$

- 📊 量化指標：
 - 最大差值 (Max Difference)：找出兩張圖在所有像素點中，差距最大的是多少。
 - 平均絕對誤差 (MAE, Mean Absolute Error)：把所有點的差距加總後除以總像素數，代表全圖平均每個點偏差了多少。

2. 為什麼照理說公式一樣，數值卻可能不同？ 🤔

OpenCV 與我們用純 NumPy 算出來的結果，通常會有 1 ~ 2 左右的微小差異。這背後有兩個核心原因：

- ✳ 定點數最佳化 vs. 浮點數運算：
NumPy 直接使用浮點數相乘（例如 $0.114 \times B$ ）；而 OpenCV 底層為了追求極致速度，會把小數轉換為整數定點數（例如乘以 $2^{14} = 16384$ ），用整數加法和位元位移（Bit Shift）來計算，最後再除回去。
- 📏 捨入方式不同 (Rounding)：
有些實作使用「四捨五入」，有些採用「直接無條件捨去 (Truncate)」，這會導致某些像素產生 ± 1 的邊界跳動。

3. 如何用程式碼找出差異？ 📄

在計算矩陣相減時，有一個初學者常踩的型態陷阱：

影像預設是 `uint8`（無號 8 位元整數，範圍 0 ~ 255）。如果發生 `0 - 1`，它不會變成 `-1`，而是會溢位 (Underflow) 變成 `255`！

因此，相減前必須先轉成有號整數：

```
Python

import numpy as np

# 1. 轉為 int 型態避免 uint8 溢位，並取絕對值
diff = np.abs(gray_opencv.astype(int) - gray_numpy.astype(int))

# 2. 計算量化統計指標
max_diff = np.max(diff)
mae = np.mean(diff)

print(f"最大像素差值: {max_diff}")
print(f"平均絕對誤差 (MAE): {mae:.4f}")
```

如果你把這段計算放進 Notebook 執行，算出來的 `max_diff` 與 `mae` 大約是多少呢？

最大像素差值：1
MAE=0.0000這是對的嗎？

這是有可能的結果。

我們可以從這兩個數字的意義來探討原因：

- 🔵 最大像素差值為 1：代表在整張影像的所有像素中，兩者算出的數值最多只差了 1（例如一個是 120，另一個是 121），沒有任何一個像素差距大於 1。
- 📊 MAE 顯示為 0.0000：當我們用 `:.4f` 格式化輸出時，小數點後四位如果低於 0.00005，就會顯示為 0.0000。如果一張影像有數百萬個像素，但其中只有極少數的像素差了 1，平均下來數值就會非常微小。

想確認到底有多少個像素不同，我們可以做進一步的檢驗。

你可以試著在 Notebook 執行下面這行程式碼，看看實際上發生差異的像素比例是多少：

```
Python

# 計算有差異的像素總數
diff_count = np.sum(diff > 0)
total_pixels = diff.size
diff_ratio = (diff_count / total_pixels) * 100

print(f"不同像素數量: {diff_count} / {total_pixels}")
print(f"差異比例: {diff_ratio:.4f}%")
print(f"未格式化的精確 MAE: {mae}")
```